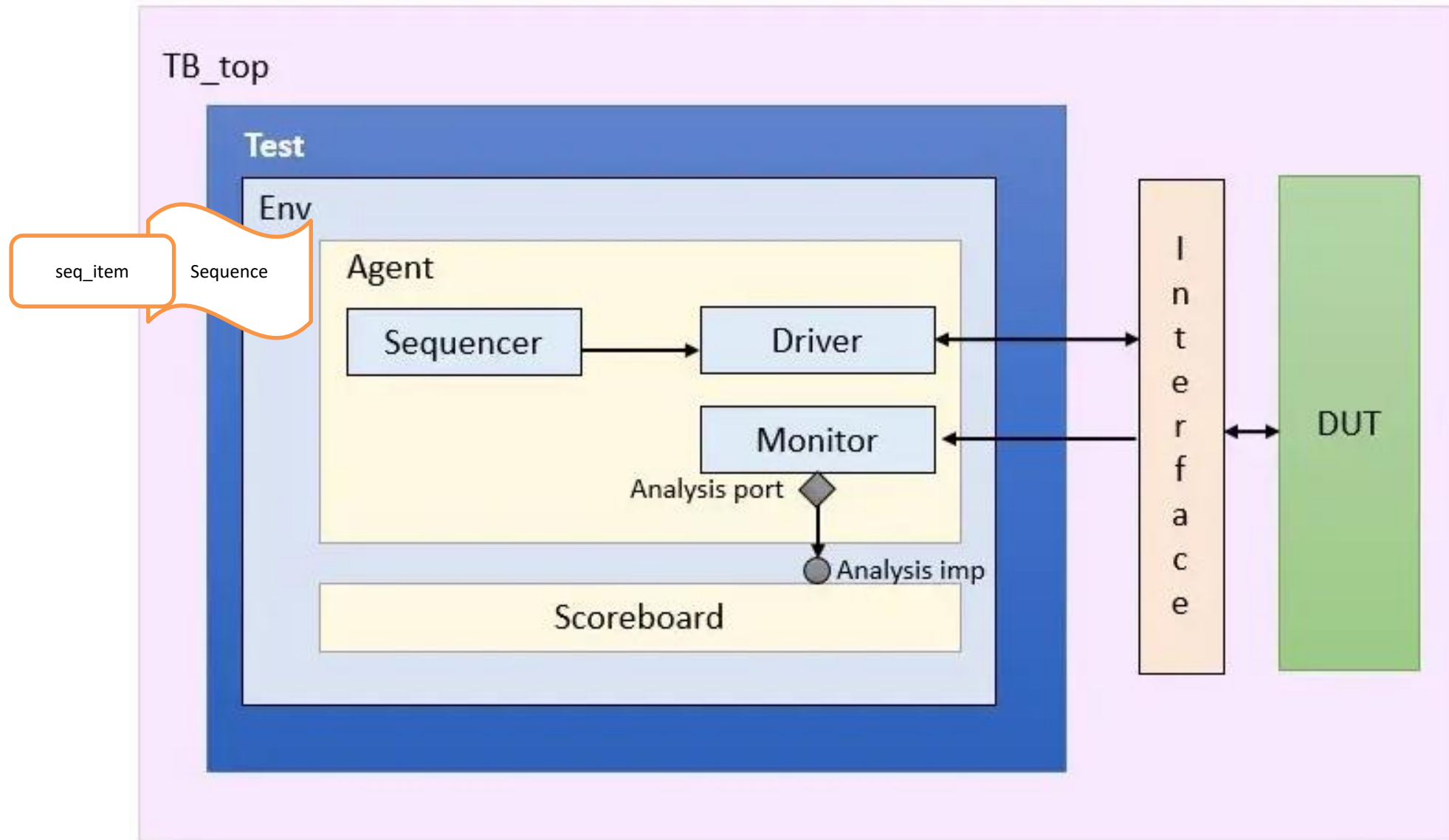


# Transaction Level Modelling (TLM) in UVM

Yasir Farooq

# Recall the UVM based Testbench



# Transaction Level Modeling in UVM

- **Transaction**

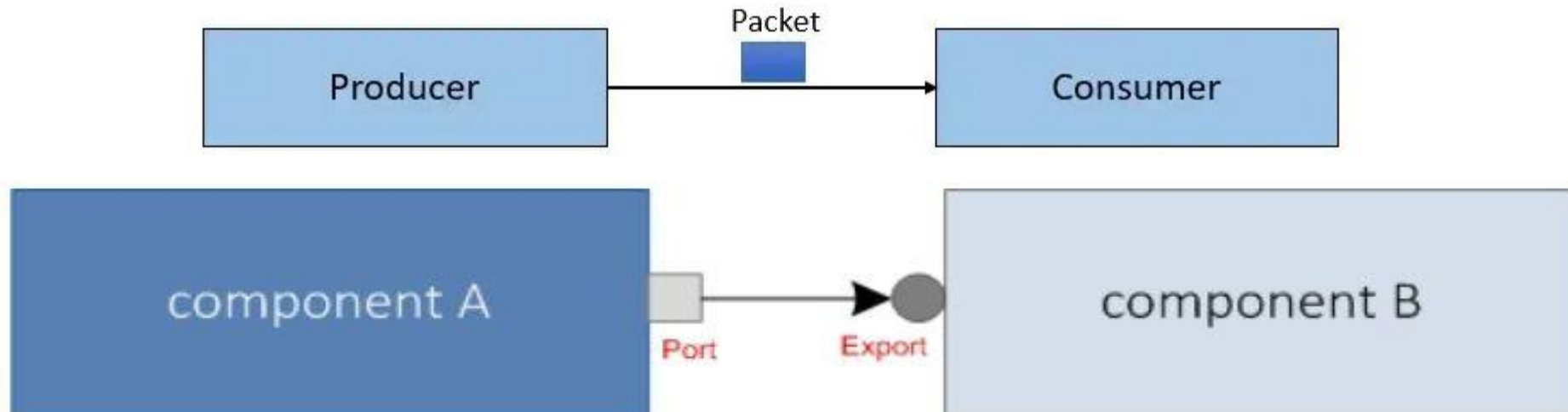
- A class object which holds the actual data flowing between uvm components via TLM interfaces
- It includes the variables and constraints on the data fields

- UVM TLM provides unidirectional and bidirectional transactions using these objects,

- TLM interfaces
- TLM ports
- TLM exports
- TLM imp ports
- TLM analysis ports
- TLM FIFOs

# UVM TLM

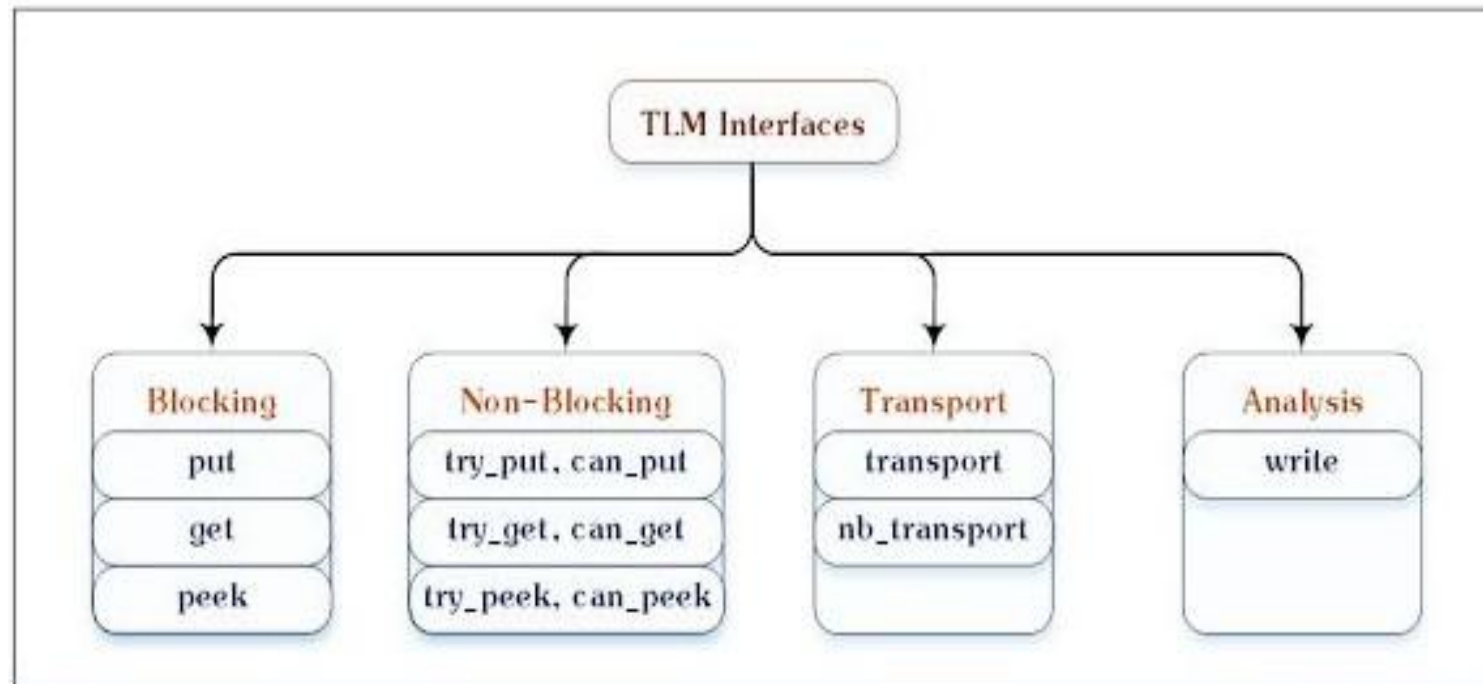
- Transaction-level communication provides reusability
  - Beauty here is, the two components are isolated from each other, user can replace component B with component C provided that component C also has TLM interface



# TLM Interface

## Transaction-Level Communication

- TLM Interface consists of methods for sending and receiving the transactions
- These methods may be blocking or non-blocking



# UVM TLM Interface

- **UVM TLM interface class declares all the methods for transaction level communication, and categorized in below three types**
  - Blocking
  - Non-Blocking
  - Combination (contains both blocking and non-blocking methods)

# Blocking

- In this type of interfaces, the methods are declared in a way that they will not proceed for next transaction until the current transaction is sent or retrieved

successfully

- put
- get
- peek
- transport

```
<port>.put(trans)
<port>.get(trans)
<port>.peek(trans)
<port>.transport(req, resp)
```

# Non-Blocking

- In this type of interfaces, the methods are declared in a way that they attempt to convey a transaction without consuming simulation time. They will return

delivery status 1 or 0

- try\_put
- can\_put
- try\_get
- can\_get
- nb\_transport
- write

```
<port>.try_put(trans)
<port>.can_put(trans)
<port>.try_get(trans)
<port>.can_get(trans)
<port>.nb_transport(req,resp)
<port>.write(trans)
```

# TLM Port and Export

TLM ports, exports, imps, analysis ports, fifos and analysis fifos are constructed using built-in UVM TLM classes

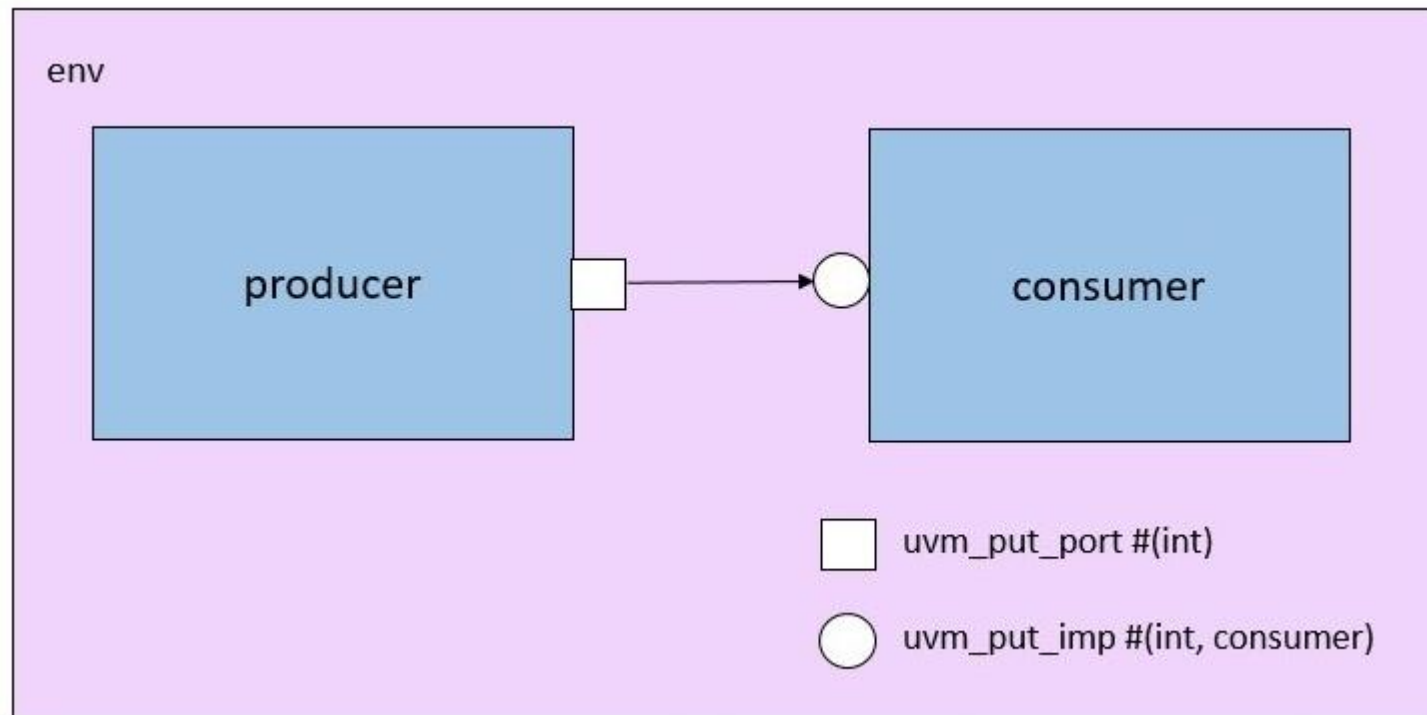
- `uvm_*_port #(T)`
- `uvm_*_export #(T)`
- `uvm_*_imp #(T,IMP)`
- `uvm_analysis_port #(T)`
- `uvm_tlm_fifo #(T)`
- `uvm_tlm_analysis_fifo#(T)`

\* may be :

- `blocking_put`
- `nonblocking_put`
- `put`
- `blocking_get`
- `nonblocking_get`
- `get`
- `blocking_peek`
- `nonblocking_peek`
- `peek`
- `blocking_get_peek`
- `nonblocking_get_peek`
- `get_peek`
- `analysis`

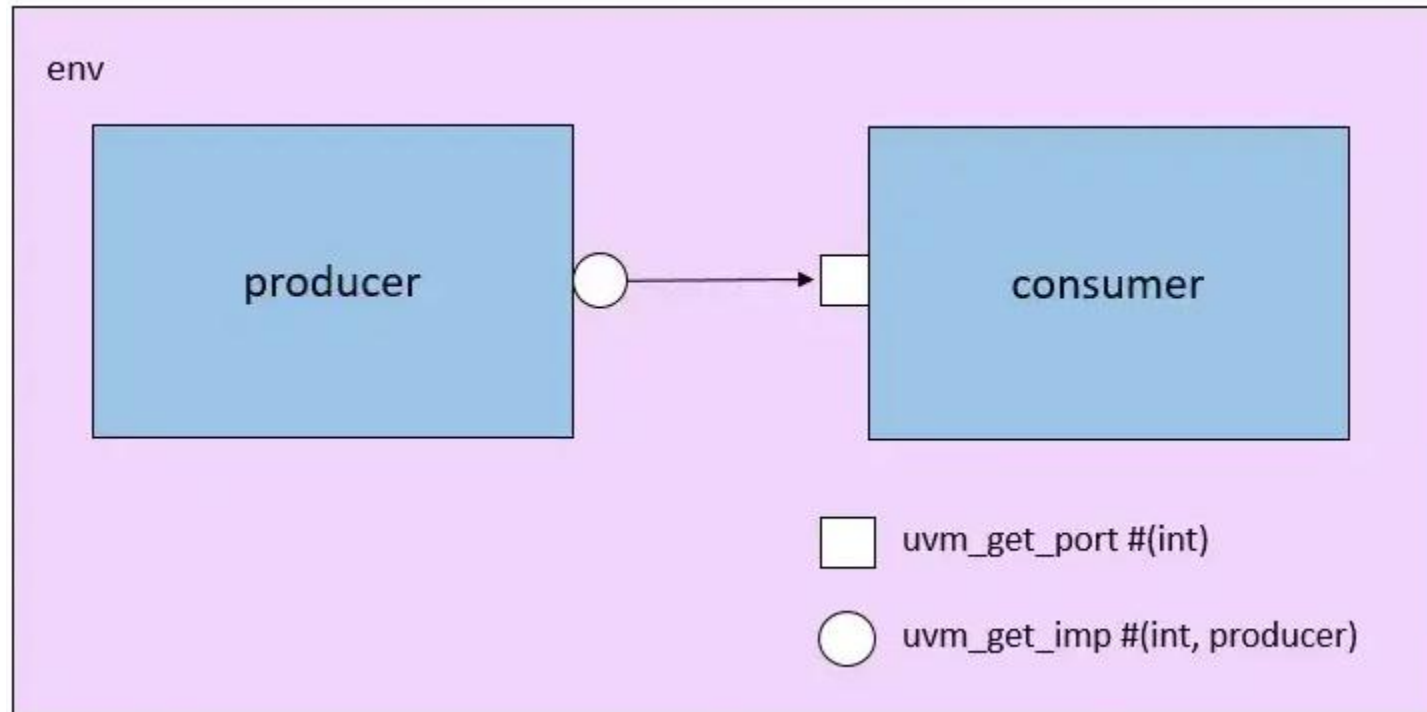
# Put Interface Example

- Producer initiates transactions using put, try\_put, and can\_put, which must be implemented in the consumer.
- [https://www.edaplayground.com/x/Xa\\_5](https://www.edaplayground.com/x/Xa_5)



# Get Interface Example

- Consumer controls the transaction flow by calling `get()`, `try_get()`, and `can_get()`, which must be implemented in the producer.
- <https://www.edaplayground.com/x/d92F> (using int)
- <https://www.edaplayground.com/x/cQUb> (uvm\_blocking\_get\_port/ imp using seq item)
- <https://www.edaplayground.com/x/qsXW> (uvm\_nonblocking\_get\_port/ imp using seq item)

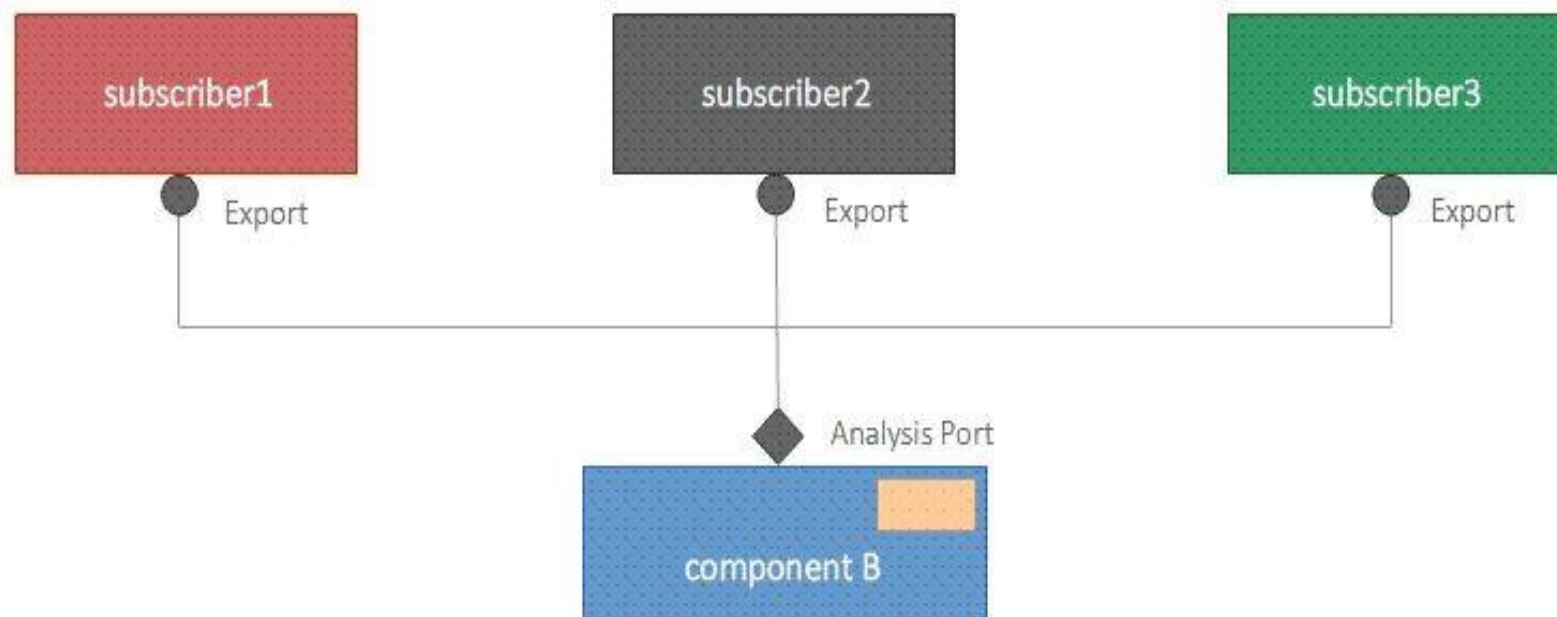


# UVM TLM Analysis Interface

- Provides a write() method for broadcasting transactions.
- Can connect to one or multiple components.
- Multiple uvm\_analysis\_port can connect to a single uvm\_analysis\_imp/export.
- uvm\_subscriber has built-in analysis\_export to receive transactions. Used for functional coverage collection in monitors.

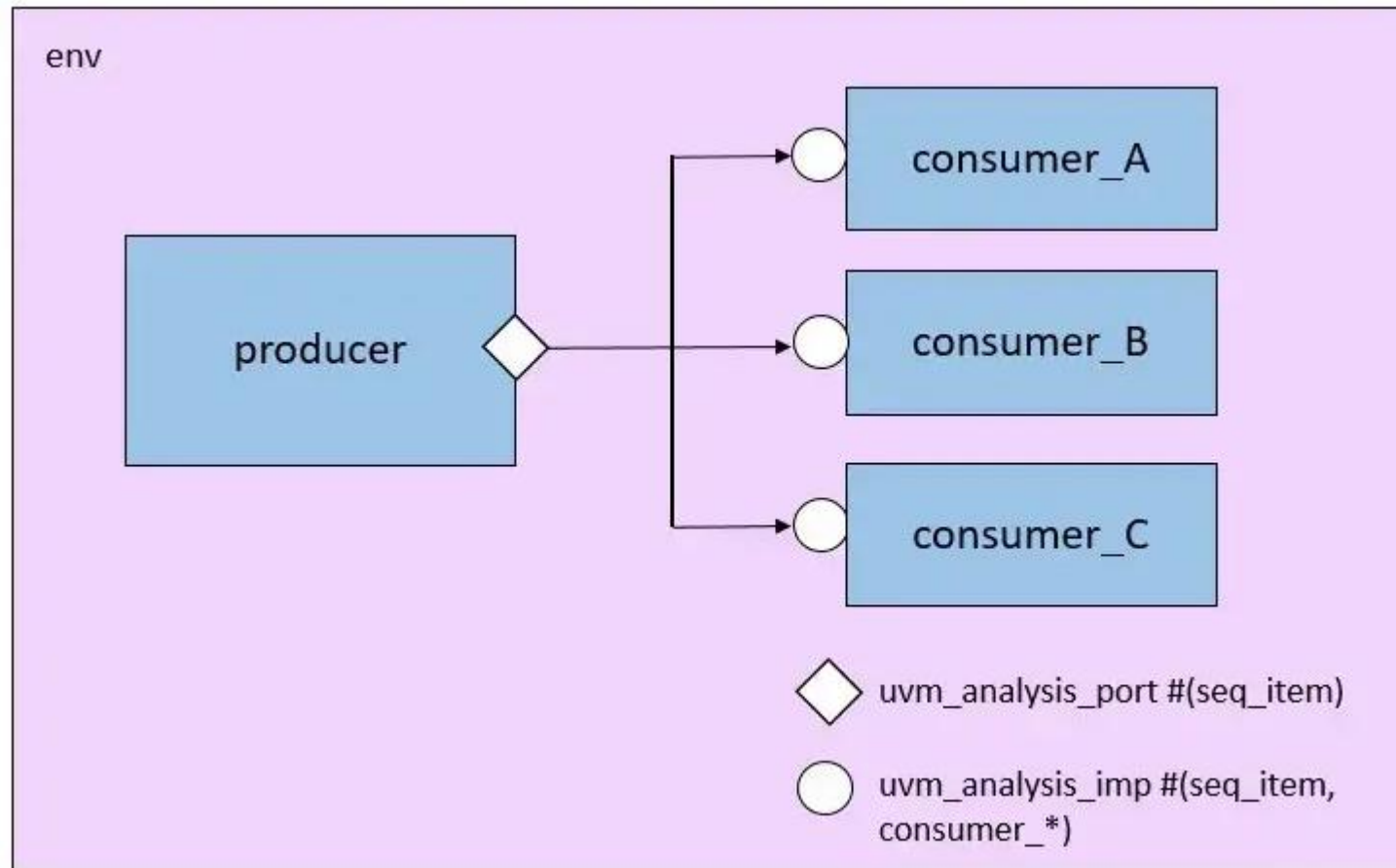
```
uvm_analysis_port #(<trans_item>) analysis_port;  
uvm_analysis_imp #(<trans_item>, <receiver_component>) analysis_imp;  
uvm_analysis_export #(<trans_item>) analysis_export;
```

# Analysis Port-Export Connection



# Analysis Port Example

<https://www.edaplayground.com/x/aVpr>



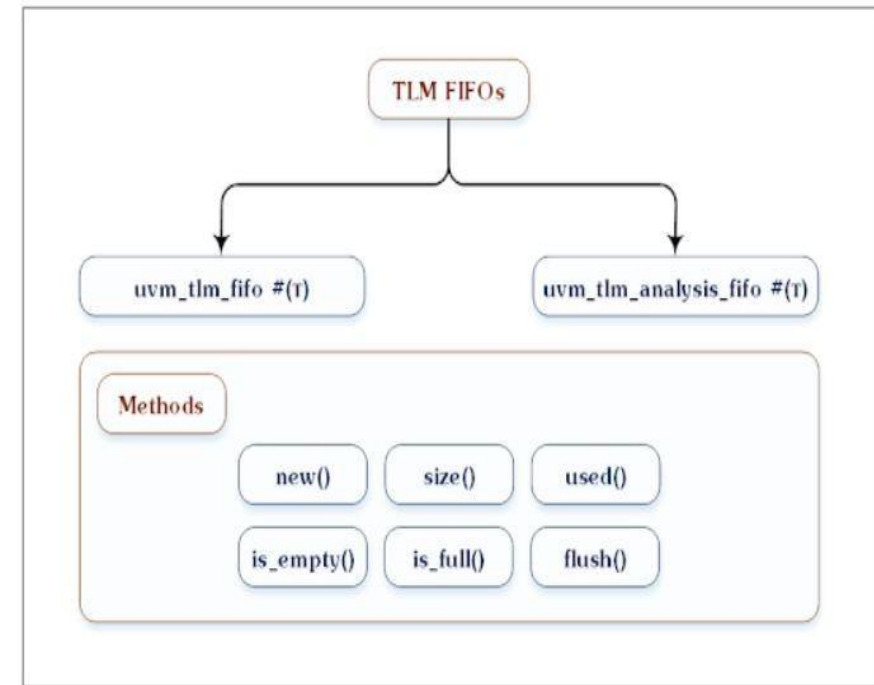
# UVM TLM FIFO

- Used to handle speed mismatches, whether the producer is faster than the consumer (preventing blocking) or the consumer is faster than the producer (buffering transactions for continuous processing).
- Stores transactions to allow independent consumer operation.
- Unlike put/get methods, no need to wait for the previous transaction to be consumed.
- Prevents blocking issues caused by speed mismatch.
- Producer doesn't wait for consumer acknowledgment before sending the next transaction.

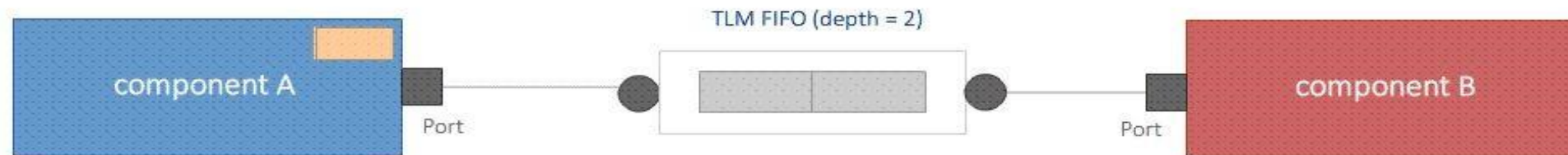
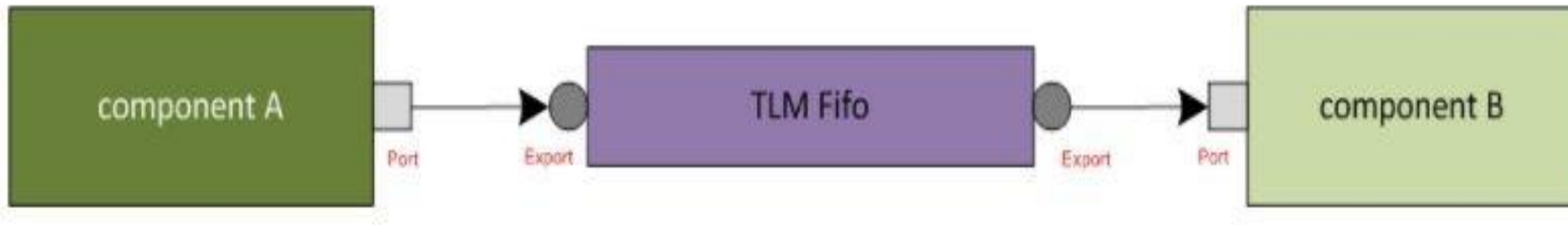
# TLM FIFO methods

|          |                                                                                                                                                                                                       |
|----------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| size     | Returns the size of TLM FIFO.<br>If a size is 0 then TLM FIFO is not bounded.                                                                                                                         |
| used     | Return number of entries put into TLM FIFO.                                                                                                                                                           |
| is_empty | No entries in TLM FIFO returns 1 else returns 0                                                                                                                                                       |
| is_full  | on TLM FIFO full, returns 1 else returns 0.<br><b>Full condition:</b> FIFO size == number of entries in FIFO.                                                                                         |
| flush    | On calling, it removes all entries from TLM FIFO<br>Thus, on calling used method, it returns 0 (<tlm_fifo>.used returns 0)<br>On calling is_empty method, it returns 1 (<tlm_fifo>.is_empty return 1) |

uvm\_tlm\_fifo

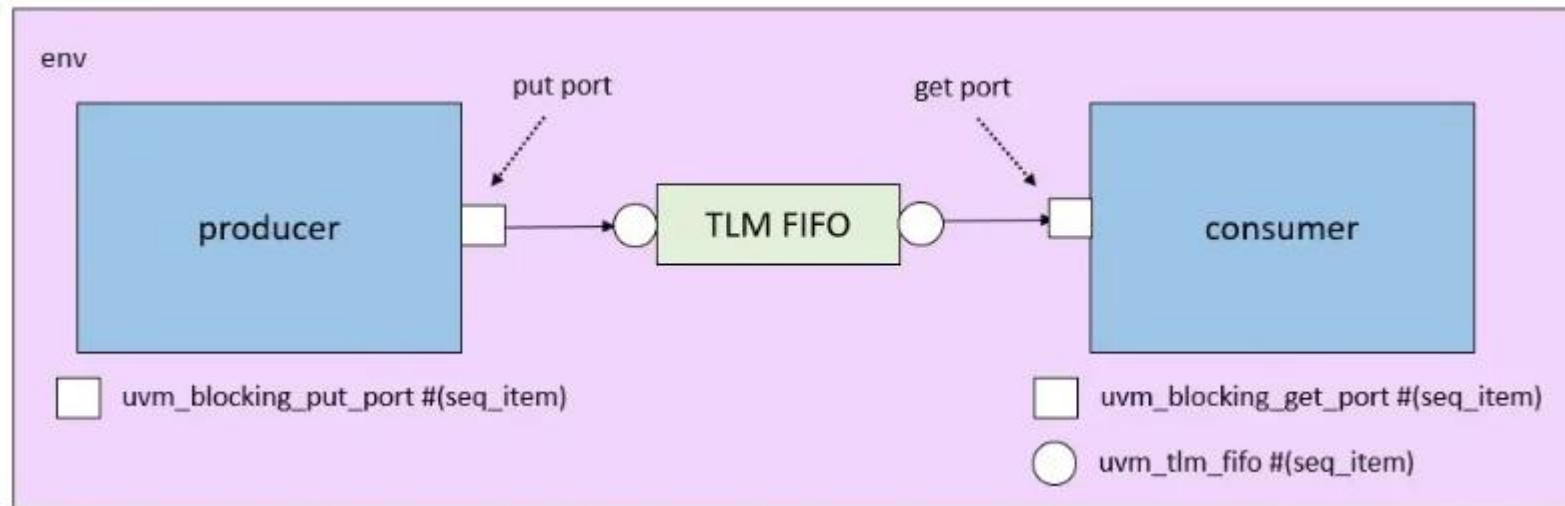


# TLM FIFO Connections



# TLM FIFO Example

<https://www.edaplayground.com/x/NLg5>



# Lab Exercise

## Lab 6 Understanding UVM TLM

To learn UVM TLM communication, ports, analysis port, blocking vs. non-blocking methods, and TLM FIFO..

Task 1:

- Recall the UVM testbench block diagram and describe where UVM TLM is utilized

Task 2:

- Analyze the code according to given block diagram

<https://www.edaplayground.com/x/Kyss>

- Where this block diagram is linked to the UVM testbench block diagram

